Verilog 语言编写规范笔记_华为

目的

本规范的目的是提高书写代码的可读性，可修改性，可重用性，优化代码综合和仿真的结果，指导设计工程师使用 Verilog HDL 规范代码和优化电路，规范化公司的 ASIC 设计，从而做到：

- 逻辑功能正确
- 可快速仿真
- 综合结果最优（ 如果是 hardware model）
- 可读性较好

范围

本规范涉及 Verilog HDL 编码风格，编码中应注意的问题， Testbench 的编码等。本规范适用于 Verilog model 的任何一级 RTL（behavioral, gate_level），也适用于出于仿真综合或二者结合的目的而设计的模块。

定义

- **Verilog HDL** ：Verilog 硬件描述语言
- **FSM** ：有限状态机
- **伪路径**：静态时序分析 STA 认为是时序失败，而设计者认为是正确的路径。

引用标准和参考资料

下列标准包含的条文 通过在本标准中引用而构成本标准的条文 在标准出版时 所示版本均为有效 所有标准都会被修订 使用本标准的各方应探讨 使用下列标准最新版本的可能性。

规范内容

一、命名规则

- 用有意义而有效的名字

  有效的命名有时并不是要求将功能描述出来，如

  For (I = 0; I < 1024; I = I + 1)

  Mem[I] <= #1 32'b0;

  For 语句中的循环指针I 就没必要用loop_index作为指针名。

- 用连贯的缩写

  长的名字对书写和记忆会带来不便，甚至带来错误。采用缩写时应注意同一信号在模块中的一致性。缩写的例子如下：

  Addr address

  Pntr pointer

  Clk clock

  Rst reset

- 用最右边的字符下划线表示低电平有效，高电平有效的信号不得以下划线表示，短暂的引擎信号建议采用高有效。

  如：***Rst_ ，Trdy_，Irdy_， Idsel.***

- 大小写原则

  名字一般首字符大写，其余小写（但parameter, integer 定义的数值名可全部用大写），两个词之间要用下划线连接。

  如：***Packet_addr, Data_in, Mem_wr Mem_ce_***

- 全局信号名字中应包含信号来源的一些信息。

  如：D_addr[7:2]，这里的“D”指明了地址是解码模块(Decoder module)中的地址。

- 同一信号在不同层次应保持一致性。
- 自己定义的常数、类型等用大写标识

  如：**parameter CYCLE=100；**

- 避免使用保留字

  如：***in，out，x，z***等不能够做为变量、端口或模块名

- 添加有意义的后缀，使信号名更加明确，常用的后缀如下：

后缀	意义
_Clk	时钟信号
_next	寄存前的信号
_z	连到三态输出的信号
_f	下降沿有效的寄存器
_xi	芯片原始输入信号
_xo	芯片原始输出信号
_xod	芯片的漏极开路输出
_xz	芯片的三态输出
-xbio	芯片的双向信号

二、Modules

1. **顶层模块应只是内部模块间的互连**。除了内部的互连和模块的调用外，尽量避免再做逻辑，如不能再出现对 reg 变量赋值等，这样做的目的是为了更有效的综合。因为在顶层模块中出现中间逻辑， Synopsys 的 design compiler 就不能把子模块中的逻辑综合到最优。

2. 每一个模块应在开始处注明文件名、功能描述、引用模块、设计者、设计时间及版权信息等。

如：
```
/* ==========================================*\
   Filename  : RX_MUX.v
   Author    :
   Description :
   Called by  : Top module
   Revision History  : 99-08-01
                       Revision 1.0
   Email  : M@sz.huawei.com.cn
   Company  : Huawei Technology .Inc
   Copyright(c) 1999, Huawei Technology Inc, All right reserved
\* ==========================================*/
```

3. 不要对 Input 进行驱动，在 module 内不要存在没有驱动的信号， 更不能在模块端口中出现没有驱动的输出信号， 避免在仿真或综合时产生 warning， 干扰错误定位。

4. 每行应限制在 80 个字符以内，以保持代码的清晰、美观和层次感。一条语句占用一行，如果较长（超出 80 个字符），则要换行。

5. 电路中调用的 module 名用 Uxx 标示。向量大小表示要清晰， 采用基于名字（name_based）的调用而非基于顺序的（order_based）。

```
Instance      UInstance2(
              .DataOut        (DOUT        ),
              .DataIn         (DIN         ),
              .Cs_                  (Cs_         )
              );
```

6. **用一个时钟的上沿或下沿采样信号，不能一会儿用上沿，一会儿用下沿。**如果既要用上沿又要用下沿，则应分成两个模块设计。建议在顶层模块中对 Clock 做一非门，在层次模块中如果要用时钟下沿就可以用非门产生的 Posedge Clk_ ，这样的好处是在整个设计中采用同一种时钟沿触发，有利于综合。（基于时钟的综合策略）

7. 在模块中增加注释。

8. Module 名要用大写标示，且应与文件名保持一致。

```
如：Module  DFF_ASYNC_RST(
                Reset,
                Clk,
                Data,
                Qout
            );
```

9. 严格芯片级模块的划分。

只有顶层包括 IO 引脚(pads) ，中间层是时钟产生模块、JTAG、 芯片的内核(CORE)，这样便于对每个模块加以约束仿真，对时钟也可以仔细仿真。

10. 模块输出寄存器化

对所有模块的输出加以寄存（如图 1 ），使得输出的驱动强度和输入的延迟可以预测，从而使得模块的综合过程更简单。输出驱动的强度都等于平均的触发器驱动强度。

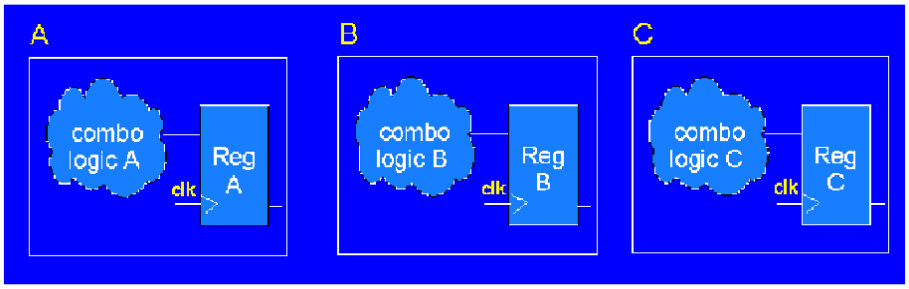


图 1：模块输出寄存器化

11. 将关键路径逻辑和非关键路径逻辑放在不同模块。

保证 DC 可以对关键路径模块实现速度优化，而对非关键路径模块实施面积优化。在同一模块 DC 无法实现不同的综合策略。

12. **将相关的组合逻辑放在同一模块**。有助于 DC 对其进行优化，因为 DC 通常不能越过模块的边界来优化逻辑。

三、Net and Register

1. 一个 reg 变量只能在一个 always 语句中赋值。

2. 向量有效位顺序的定义一般是从大数到小数。推荐 Data[4：0]这种格式的定义。

3. 对 net 和 register 类型的输出要做声明 （在 PORT 中）。如果一个信号名没做声明 Verilog 将假定它为一位宽的 wire 变量。

4. 线网的多种类型。寄存器的类型。

四、Expressions

1. 用括号来表示执行的优先级，尽管操作符本身有优先顺序但用括号来表示优先级对读者更清晰，更有意义。

If ((alpha < beta) && (gamma >= delta)).... 比下面的表达更合意

If (alpha < beta && gamma >= delta)...

2. 用一个函数(function)来代替表达式的多次重复。

五、IF 语句

1. 向量比较时，比较的向量要相等。
2. 每一个 If 都应有一个 else 和它相对应。
3. 应注意 If ..else if ...else if ...else 的优先级。
4. 如果变量在 If-else 或 case 语句中做非完全赋值，则应给变量一个缺省值。

六、case 语句

1. **case 语句通常综合成一级多路复用器（图 2 的右边部分），而 if-then-else 则综合成优先编码的串接的多个多路复用器（如图 2 的左边部分）。** 通常，使用 case 语句要比 if 语句快，优先编码器的结构仅在信号的到达有先后时使用。条件赋值语句也能综合成多路复用器，而 case 语句仿真要比条件赋值语句快。

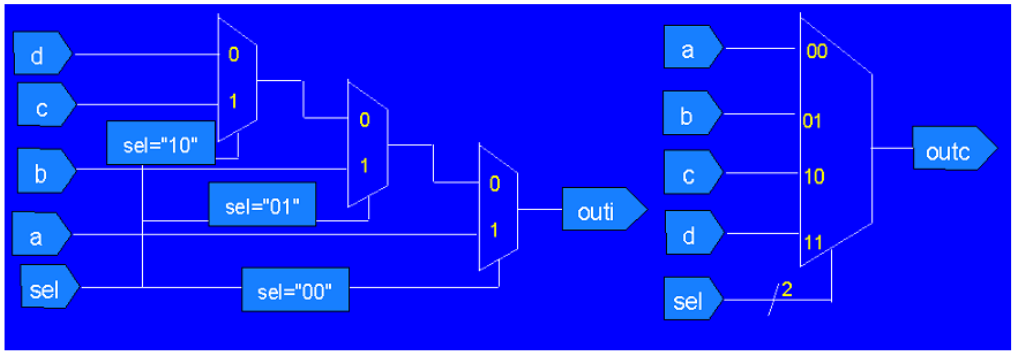


图 2：case 语句和 if-then-else 的综合结果

2. 所有的 Case 应该有一个 default case ，允许空语句

Default : ;

七、Writing functions

1. 在 function 的最后给 function 赋值。
2. **函数中避免使用全局变量**，否则容易引起 HDL 行为级仿真和门级仿真的差异。

   最好直接在端口加以定义 。注意，函数与任务的调用均为静态调用。

八、Assignment

1. Verilog 支持两种赋值：过程赋值(procedural) 和连续赋值(continuous) 。过程赋值用于过程代码（initial, always, task or function)中给 reg 和 integer 变量、time\realtime、real 赋值，而连续赋值一般给 wire 变量赋值。
2. Always @(敏感表)，敏感表要完整，如果不完整，将会引起仿真和综合结果不一致。
3. Assign/deassign 仅用于仿真加速，仅对寄存器有用。
4. Force/release 仅用于 debug，对寄存器和线网均有用。
5. 避免使用 Disable。
6. 对任何 reg 赋值用非阻塞赋值代替阻塞赋值，reg 的非阻塞赋值要加单位延迟，但异步复位可加可不加。

九、Combinatorial Vs Sequential Logic

1. 如果一个事件持续几个时钟周期，设计时就用时序逻辑代替组合逻辑。
2. 内部总线不要悬空。在 default 状态，要把它上拉或下拉。

```
Wire        OE_default;
Assign      OE_default = !(oe1 | oe2 | oe3);
Assign      bus[31:0] = oe1 ? Data1[31:0] :
                        oe2 ? Data2[31:0] :
                        oe3 ? Data3[31:0] :
                        oe_default ? 32'h0000_0000 :
                        32'hzzzz_zzzz;
```

十、Macros

1. 为了保持代码的可读性，常用“`define“做常数声明。

2. 把“`define”放在一个独立的文件中。

参数 parameter 必须在一个模块中定义，不要传替参数到模块（仿真测试向量例外），“`define”可以在任何地方定义，要把所有的“`define”定义在一个文件中，

在编译原代码时首先要把这个文件读入。如果希望宏的作用域仅在一个模块中，就用参数来代替。

十一、Comments

1. 对更新的内容要做注释。
2. 在语法块的结尾做标记。

```
//style  1
   If (~OE_ && (state != PENDING)) begin
       ....
   End  // if enable = = ture and ready

//style  2  --- identical lables on begin and end
   If (~OE_ && (state != PENDING)) begin    //drive data
       ....
   End     //drive data
//  Comment end<unit> with the name of the <unit>
   Function  Calcparity; //Data, ParityErr
       ....
   Endfunction     // Calcparity
```

3. 每一个模块都应在模块开始处做模块级的注释（参考前面标准模块头）。
4. 在模块端口列表中出现的端口信号，都应做简要的功能描述。

十二、FSM

1. VerilogHDL 状态机的状态分配

VerilogHDL 描述状态机时，必须由 parameter 分配好状态，这与 VHDL 不同，VHDL 状态机状态可以在综合时分配产生。

2. 组合逻辑和时序逻辑分开，用不同的进程。

组合逻辑包括状态译码和输出，时序逻辑则是状态寄存器的切换。

3. 必须包括对所有状态都处理，不能出现无法处理的状态，使状态机失控。

4. Mealy 机的状态和输入有关，而 Moore 机的状态转换和输入无关。

十三、代码编写中容易出现的问题

1. 在 for-loop 中包括不变的表达式，浪费运算时间。
2. 资源共享问题

条件算子中不存在资源共享，如 z = (cond) ? (a + b) : (c + d); 必须使用两个加法器;

而等效的条件 if-then-else 语句则可以资源共享，如

```
if (Cond)
    z = a + b;
else
    z = c + d;
```

只要加法器的输入端复用，就可以实现加法器的共享，使用一个加法器实现。

3. 由于组合逻辑的位置不同而引起过多的触发器综合。
4. 谨慎使用异步逻辑。
5. 对组合逻辑的描述有多种方式，其综合结果是等效的。
6. 考虑综合的执行时间

通常会推荐将模块划分得越小越好，事实上要从实际的设计目标、面积和时序要求出发。好的时序规划和合适的约束条件要比电路的大小对综合时间的影响要大。

最好在设计阶段就做好时序规划，通过综合的约束 scripts 来满足时序规划。从代码设计讲，500~5000 行的长度是合适的。

7. 避免点到点的例外

所谓点到点例外（Point-to-point exception），就是从一个寄存器的输出到另一个寄存器的输入的路径不能在一个周期内完成。多周期路径就是其典型情况。

避免使用多周期路径，如果确实要用，应将它放在单独一个模块，并且在代码中加以注释。

8. 避免伪路径(False path)

伪路径是那些静态时序分析（STA） 认为是时序失败，而设计者认为是正确的路径。通常会人为忽略这些 warning ，但如果数量较多时，就可能将其他真正的问题错过了。

9. 避免使用 Latch

使用 Latch 必须有所记录，可以用 All_registers -level_sensitive 来报告设计中用到的 Latch。不希望使用 Latch 时，应该对所有输入情况都对输出赋值，或者将条件赋值语句写全，如在 if 语句最后加一个 else，case 语句加 defaults。当你必须使用 Latch 时，为了提高可测性，需要加入测试逻辑。

10. 避免使用门控时钟

使用**门控时钟**(Gated clock)不利于移植，可能引起毛刺，带来时序问题，同时对扫描链的形成带来问题。门控钟在低功耗设计中要用到，但通常不要在模块级代码中使用。可以借助于 Power compiler 来生成，或者在顶层产生。

11. 避免使用内部产生的时钟

在设计中最好使用同步设计。如果要使用内部时钟，可以考虑使用多个时钟。

12. 避免使用内部复位信号

模块中所有的寄存器最好同时复位。如果要使用内部复位，最好将其相关逻辑放在单独的模块中，这样可以提高可阅读性。

13. 如果确实要使用内部时钟，门控时钟，或内部的复位信号，将它们放在顶层。

将这些信号的产生放在顶层的一个独立模块，这样所有的子模块分别使用单一的时钟和复位信号。一般情况下内部门控时钟可以用同步置数替代。例如：

```
module COUNT (Reset， Enable， Clk， Qout)；
input Reset， Enable， Clk；

output [2:0] Qout；
reg [2:0] Qout；
```

```
module COUNT (Reset， Enable，
                Clk， Qout)；
input Reset， Enable， Clk；

output [2:0] Qout；
reg [2:0] Qout；
```

```
wire GATED_Clk = Clk & Enable;

always @(posedge GATED_Clk or posedge Reset)
begin
  if (Reset) begin
    Qout = 1'b0;
  end
  else begin
    if (Qout == 3'd7) begin
      Qout= 1'b0;
    end
  else begin
    Qout = Qout + 1'b1;
  end
end
end
endmodule
```

```
always @(posedge Clk) begin

  if (Reset) begin
      Qout = 1'b0;
  end
  else if (Enable == 1'b1) begin

      if (Qout == 3'd7) begin

        Qout = 1'b0;
      end
    else begin
        Qout = Qout + 1'b1;

      end
  end
end
endmodule
```

参考：大规模逻辑设计指导书_华为